

Name - VINAY Email - vinay.23FE10CSE00607@muji.manipal.edu Section - I Batch - 2

To construct a Naive Bayes classifier

Load the Dataset and Add Headers

```
import pandas as pd
import numpy as np

# Giving the columns names so the data actually makes sense
column_labels = [
    'age', 'workclass', 'fnlwgt', 'education', 'education_num',
    'marital_status', 'occupation', 'relationship', 'race', 'sex',
    'capital_gain', 'capital_loss', 'hours_per_week', 'native_country', 'income'
]

# Loading the data and stripping out extra spaces
my_data = pd.read_csv('adult.data', names=column_labels, sep=',', skipinitialspace=True)

# Just checking the first few rows to make sure it loaded right
print(my_data.head())
```

	age	workclass	fnlwgt	education	education_num	\
0	39	State-gov	77516	Bachelors	13	
1	50	Self-emp-not-inc	83311	Bachelors	13	
2	38	Private	215646	HS-grad	9	
3	53	Private	234721	11th	7	
4	28	Private	338409	Bachelors	13	

	marital_status	occupation	relationship	race	sex	\
0	Never-married	Adm-clerical	Not-in-family	White	Male	
1	Married-civ-spouse	Exec-managerial	Husband	White	Male	
2	Divorced	Handlers-cleaners	Not-in-family	White	Male	
3	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	
4	Married-civ-spouse	Prof-specialty	Wife	Black	Female	

	capital_gain	capital_loss	hours_per_week	native_country	income
0	2174	0	40	United-States	<=50K
1	0	0	13	United-States	<=50K
2	0	0	40	United-States	<=50K
3	0	0	40	United-States	<=50K
4	0	0	40	Cuba	<=50K

Cleaning the "Messy" Data

Column Headers: We added names to the columns because the raw file didn't have titles, and we removed extra spaces so the computer could read the text correctly.

Handling Missing Info: We found all the rows with "?" marks and deleted them to make sure the model only learned from complete and accurate data.

```
# The dataset uses '?' for missing info, so let's swap those to NaN
my_data.replace('?', np.nan, inplace=True)

# Checking how many rows have missing values
print("Rows before cleaning:", len(my_data))
my_data.dropna(inplace=True)
print("Rows after dropping unknowns:", len(my_data))
```

```
Rows before cleaning: 32561
Rows after dropping unknowns: 30162
```

: Converting Text to Numbers (Encoding)

Text to Numbers: Since the algorithm only understands math, we converted categories like "Education" and "Job" into numbers using a tool called LabelEncoder.

Formatting: We made sure every piece of information was in a numeric format so the Naive Bayes classifier could calculate probabilities for the final prediction.

```

from sklearn.preprocessing import LabelEncoder

# I'll create an encoder object
label_transformer = LabelEncoder()

# These are all the columns that are text-based
categorical_stuff = [
    'workclass', 'education', 'marital_status', 'occupation',
    'relationship', 'race', 'sex', 'native_country', 'income'
]

# Loop through and transform them all into numbers
for col in categorical_stuff:
    my_data[col] = label_transformer.fit_transform(my_data[col])

# Now the whole dataframe should be numeric
print(my_data.head())

```

```

   age  workclass  fnlwgt  education  education_num  marital_status  \
0   39         5    77516         9             13             4
1   50         4    83311         9             13             2
2   38         2   215646        11             9              0
3   53         2   234721         1             7              2
4   28         2   338409         9             13              2

   occupation  relationship  race  sex  capital_gain  capital_loss  \
0           0              1    4    1          2174             0
1           3              0    4    1              0             0
2           5              1    4    1              0             0
3           5              0    2    1              0             0
4           9              5    2    0              0             0

   hours_per_week  native_country  income
0              40              38        0
1              13              38        0
2              40              38        0
3              40              38        0
4              40               4        0

```

Splitting the Data

The 80/20 Split: We separated the data into two groups—one for the model to study (80%) and one for us to use as a "final exam" (20%).

Unseen Data: By testing the model on data it had never seen before, we proved that it actually learned how to predict income rather than just memorizing the file.

```

from sklearn.model_selection import train_test_split

# 'income' is what we are trying to predict (the target)
X = my_data.drop('income', axis=1)
y = my_data['income']

# Splitting it up
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training on {len(X_train)} rows and testing on {len(X_test)} rows.")

Training on 24129 rows and testing on 6033 rows.

```

Training and Running the Model

Naive Bayes Model: We used the Gaussian Naive Bayes algorithm to look at all the features (like age and hours worked) and predict if someone earns over \$50k.

Performance Check: Instead of just looking at the final score, we used a classification report to see exactly how good the model was at catching both high and low earners.

```

from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

# Initialize the model
income_classifier = GaussianNB()

# Teach the model using our training data
income_classifier.fit(X_train, y_train)

```

```
# Use the model to predict the test set
predictions = income_classifier.predict(X_test)

# Let's see how we did
final_accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {round(final_accuracy * 100, 2)}%")
print("\nFull Report:")
print(classification_report(y_test, predictions))
```

Accuracy: 79.08%

Full Report:

	precision	recall	f1-score	support
0	0.80	0.95	0.87	4503
1	0.69	0.32	0.44	1530
accuracy			0.79	6033
macro avg	0.75	0.64	0.65	6033
weighted avg	0.77	0.79	0.76	6033

Start coding or [generate](#) with AI.